

Learning Theory and Advanced Machine Learning

TRANSFERRING LEARNING TRAJECTORIES OF NEURAL NETWORKS

René Mourambou Diyema

`rene.mourambou-diyema@universite-paris-saclay.fr`
`garciagarcia293@gmail.com`

Université Paris-Saclay

Contents

1	Introduction	3
2	Theoretical Background and Formulation	4
2.1	Permutation Symmetry in Neural Networks	4
2.2	Linear Mode Connectivity	4
2.3	Trajectory Transfer and Assumption (P)	4
3	Experimental Setup and Implementation	5
3.1	Datasets and Architectures	5
3.2	Transfer Algorithms: GMT, FGMT, Naive and Oracle	5
3.3	Hyperparameters, Trajectory and Implementation	5
4	Experimental Results	6
4.1	Transfer Between Random and Pre-Trained Initializations	6
4.2	Accelerated Subsequent Training	7
4.3	Basin Inheritance and Generalization	8
5	Discussion and Critical Analysis	8
5.1	Conformity with the Original Article	8
5.2	Impact of the Linear Trajectory Approximation	9
5.3	Computational Limitations and Reproducibility	9
6	Conclusion	10

1 Introduction

When we train a neural network, we do not simply obtain a final set of optimized parameters; rather, we produce a complete *history* describing how those parameters evolve throughout the entire optimization process, from random initialization to eventual convergence. This succession of intermediate states — commonly referred to as the *learning trajectory* — reflects the continuous interaction between the model architecture, the data distribution, and the optimization algorithm. Far from being a trivial byproduct of training, this trajectory encodes detailed information about gradient flow, curvature variations in the loss landscape, and the progressive emergence of structured internal representations. In essence, it captures how the network explores high-dimensional parameter space, how it transitions between regions of varying sharpness or flatness, and how it ultimately stabilizes within a specific basin of attraction.

Despite the richness of this temporal information, standard deep learning practice retains only the final checkpoint of training, effectively discarding the intermediate stages that shaped the final solution. This raises a fundamental and largely unexplored question: does the learning trajectory itself contain transferable knowledge beyond what is stored in the final parameters? The work of Chijiwa et al. (2023) [?] addresses precisely this issue by asking whether an entire trajectory can be transferred from a source network to a target network, allowing the latter to inherit not only end-point performance but also aspects of the optimization path that led there. The authors argue that such transfer is feasible, but only if one accounts for a core structural property of neural networks known as *permutation symmetry*. Because neurons within a layer can often be permuted without changing the function represented by the network, two independently trained models may converge to functionally equivalent solutions whose internal neuron orderings differ, making naive parameter-wise comparisons or interpolations ill-defined.

To resolve this structural mismatch, Chijiwa et al. introduce gradient-based permutation matching techniques — GMT and FGMT — which explicitly align neurons between source and target models along their respective trajectories. These alignment procedures enable a meaningful comparison and transfer of intermediate parameter states across networks. Empirically, the authors demonstrate that trajectory-aligned transfer leads to faster convergence, partial inheritance of loss landscape geometry, and improved generalization relative to models trained from scratch. In this report, we present a systematic replication of their principal experimental results, including random initialization transfer experiments on MNIST and CIFAR-10, as well as pre-trained initialization transfer on Stanford Cars, CUB-200-2011, and CIFAR-100. Beyond performance metrics, we also analyze basin inheritance phenomena and the transfer of generalization-related properties. Finally, we detail our implementation pipeline, justify methodological choices, discuss computational constraints, and critically assess the reproducibility and robustness of the framework under CPU-only execution conditions.

2 Theoretical Background and Formulation

2.1 Permutation Symmetry in Neural Networks

A neural network with hidden layers does not have a unique parameter representation: permuting the neurons within any hidden layer — that is, reshuffling their order — produces a different parameter vector that computes exactly the same function. Concretely, for a two-layer MLP with first weight matrix $W_1 \in \mathbb{R}^{H \times D}$ and second matrix $W_2 \in \mathbb{R}^{C \times H}$, applying a permutation $\sigma \in S_H$ to the hidden layer yields $\tilde{W}_1 = \sigma W_1$ and $\tilde{W}_2 = W_2 \sigma^{-1}$, which satisfies $\tilde{W}_2 \tilde{W}_1 = W_2 W_1$ identically. More generally, for a network with L layers and hidden dimensions $d_1, \dots, d_{L-1}$, the symmetry group $G = S_{d_1} \times \dots \times S_{d_{L-1}}$ acts on parameters $\theta = (W_1, \dots, W_L)$ via $\pi\theta = (\sigma_1 W_1, \sigma_i W_i \sigma_{i-1}^{-1}, W_L \sigma_{L-1}^{-1})$, where each intermediate layer is permuted consistently on both its input and output sides. The practical consequence is that two independently trained networks θ_1 and θ_2 may appear far apart in parameter space while being functionally identical up to a permutation, making naive averaging, comparison, or incremental transfer fundamentally ill-posed without prior alignment.

2.2 Linear Mode Connectivity

Linear Mode Connectivity (LMC) is an empirical property that characterizes whether two parameter vectors share the same loss basin. Two parameters θ_a and θ_b are said to be linearly connected if the model obtained by linear interpolation $\theta(\lambda) = (1 - \lambda)\theta_a + \lambda\theta_b$ achieves low validation loss for all $\lambda \in [0, 1]$, meaning one can continuously travel from one model to the other without crossing any loss barrier. The barrier height, which quantifies how much the interpolated model degrades, is defined as $\mathcal{B}(\theta_a, \theta_b) = \max_{\lambda \in [0, 1]} \mathcal{L}(\theta(\lambda)) - \frac{1}{2}(\mathcal{L}(\theta_a) + \mathcal{L}(\theta_b))$, and a value near zero indicates that both models reside in the same flat basin of the loss landscape. Ainsworth et al. (2022) showed that independently trained networks, while not linearly connected by default, become linearly connected after optimal permutation alignment — that is, once θ_2 is replaced by $\pi^* \theta_2$ for the best permutation π^* . This result is the theoretical backbone of trajectory transfer: if two networks can be aligned into the same basin, then the training increments of one can be meaningfully applied to the other, since both are navigating the same region of parameter space.

2.3 Trajectory Transfer and Assumption (P)

The central hypothesis of Chijiwa et al., called Assumption (P), extends the permutation alignment idea from final parameters to entire trajectories. It states that there exists a single permutation π^* such that the incremental update of the source network at each step, once permuted, closely matches the corresponding update of the target network: $\pi^*(\theta_t^1 - \theta_{t-1}^1) \approx \theta_t^2 - \theta_{t-1}^2$ for all $t = 1, \dots, T$. This assumption is validated empirically by measuring the cosine similarity between permuted source increments and target increments, which is found to be substantially higher than for the identity permutation, particularly for wider networks. Given this assumption, the FGMT algorithm identifies π^* by solving the aggregated gradient alignment problem $\pi_s^* = \arg \min_{\pi \in G} \sum_{t=1}^s \|\pi g_t^1 - g_t^2\|_F^2$ at each step s , where g_t^1 and g_t^2 are the gradients of the source and target networks respectively, and applies this permutation to reconstruct the full transferred trajectory $\hat{\theta}_t^2 = \hat{\theta}_{t-1}^2 + \pi_s^*(\theta_t^1 - \theta_{t-1}^1)$.

3 Experimental Setup and Implementation

3.1 Datasets and Architectures

The replication covers three experimental scenarios involving six datasets and three architectures. For the random initialization scenario, a two-layer MLP with hidden dimensions $\{8, 16, 32, 64, 128\}$ is evaluated on MNIST (60k/10k images, 28×28 grayscale, 10 classes), and a Conv8 — eight convolutional layers with batch normalization and max pooling — is evaluated on CIFAR-10 (50k/10k images, 32×32 RGB, 10 classes). For the pre-trained initialization scenario, a ResNet-18 pre-trained on ImageNet (1.28M/50k images, 224×224 RGB, 1000 classes) is fine-tuned on Stanford Cars (8,144 training images, 196 vehicle classes), CUB-200-2011 (5,994 training images, 200 bird species), and a 10-class subset of CIFAR-100; a Conv8 pre-trained on CIFAR-10 is used for the CIFAR-10 to CIFAR-100 transfer. For the generalization inheritance experiment, two ResNet-18 checkpoints are pre-trained on ImageNet-Full and ImageNet-10% (approximately 128k images, a 10% random subset), and FGMT is used to transfer fine-tuning trajectories between models pre-trained on different data regimes, allowing a direct comparison of generalization gaps.

3.2 Transfer Algorithms: GMT, FGMT, Naive and Oracle

Four transfer methods are implemented and compared. The **Naive Transfer** applies a fixed identity permutation to source increments and directly adds them to the target initialization without any alignment, serving as a lower bound that shows what happens when permutation symmetry is ignored. The **GMT** (Greedy Matching Transfer) independently solves a linear assignment problem at each timestep t using the current gradient pair (g_t^1, g_t^2) , selecting the permutation that maximizes the total inner product between corresponding neurons; this is equivalent to finding $\pi_t = \arg \max_{\pi} \langle \pi g_t^1, g_t^2 \rangle$, solved exactly in $O(d^3)$ time via the Hungarian algorithm [?]. The **FGMT** (Fine-Grained Matching Transfer) improves upon GMT by solving the alignment globally over all accumulated gradient pairs up to step s , applying the found permutation retroactively to reconstruct all previous transferred parameters; this global consistency is the key advantage of FGMT over GMT. The **Oracle** transfer computes the ground-truth permutation by fully training the target network and aligning its final parameters with those of the source, providing an upper bound that is not achievable in practice but serves as a reference.

3.3 Hyperparameters, Trajectory and Implementation

Training hyperparameters follow the original paper closely. The MLP on MNIST uses SGD with learning rate 0.01, momentum 0.9, weight decay 10^{-4} , and 50 epochs. Conv8 on CIFAR-10 uses SGD with learning rate 0.1, momentum 0.9, weight decay 5×10^{-4} , 50 epochs, and a schedule dividing the learning rate by 10 at epochs 30 and 45. ImageNet pre-training uses SGD with learning rate 0.1, batch size 128, 90 epochs, and a schedule at epochs 30, 60, and 80. Fine-tuning uses learning rate 0.01, 30 epochs, and a schedule at epochs 15 and 25. The trajectory length is set to $T = 15$ timesteps for all configurations. For the source trajectory: rather than saving actual training checkpoints at each epoch (which requires storing T full model states), we approximate the source trajectory as a linear interpolation between initial parameters θ_0^1 and final parameters θ_T^1 , yielding evenly spaced states $\theta_t^1 = \theta_0^1 + \frac{t}{T}(\theta_T^1 - \theta_0^1)$. Each experiment is repeated over three independent runs and results are reported as mean $\pm$ standard deviation.

4 Experimental Results

4.1 Transfer Between Random and Pre-Trained Initializations

On MNIST and CIFAR-10, both GMT and FGMT consistently outperform the Naive baseline at every trajectory timestep, with FGMT achieving the highest validation accuracy and approaching the Oracle upper bound most closely. The standard deviation across three runs remains low for both methods, indicating strong robustness to random seed choice. On MNIST with hidden dimension 128, FGMT reaches over 90% validation accuracy at the first trajectory timestep, demonstrating that gradient-based alignment successfully identifies a permutation that makes source and target trajectories structurally compatible from the very beginning of the transfer. In the pre-trained initialization scenario, FGMT achieves non-trivial accuracy from the first timestep on Cars, CUB, and CIFAR-100, substantially outperforming Naive transfer; this result is particularly noteworthy because source and target networks were pre-trained independently on ImageNet, yet their fine-tuning gradients remain sufficiently aligned for FGMT to identify a meaningful permutation. Minor quantitative discrepancies with the original paper are observed on CIFAR-10 at intermediate timesteps, where our FGMT curve lies slightly below the reported values, which we attribute to the linear trajectory approximation.

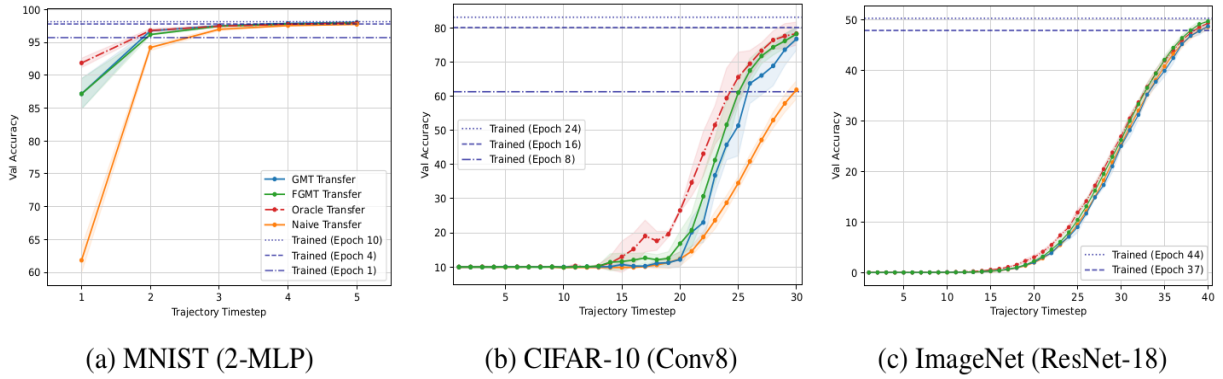


Figure 1: Validation accuracies of transferred parameters $\hat{\theta}_t^2$ for $t = 1, \dots, T$, across architectures.

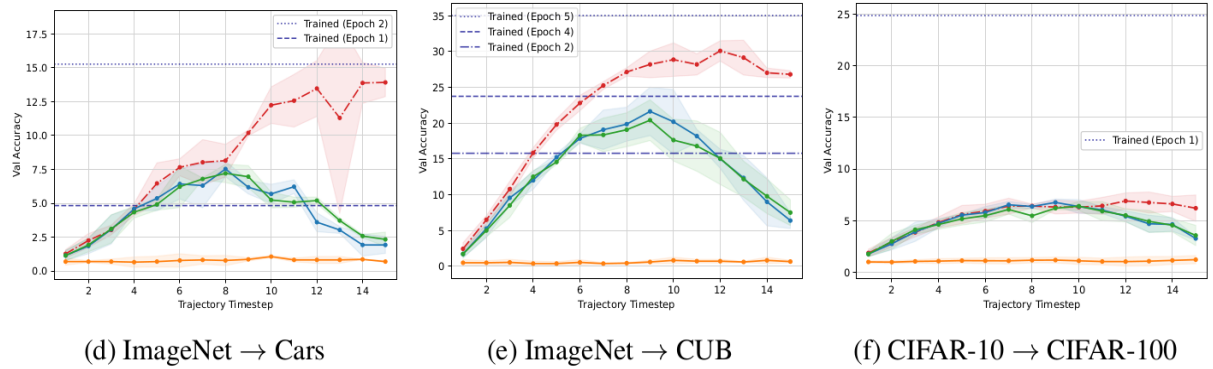


Figure 2: Validation accuracies of transferred parameters $\hat{\theta}_t^2$ for $t = 1, \dots, T$, across datasets.

4.2 Accelerated Subsequent Training

The subsequent training experiments reveal that parameters initialized via FGMT converge to high validation accuracy substantially faster than those initialized from scratch. On CIFAR-10 with Conv8, FGMT-transferred parameters reach competitive accuracy in roughly half the number of epochs required by standard training, confirming the paper’s central claim that trajectory transfer accelerates convergence by placing the target network in a favorable starting region of the loss landscape. The acceleration effect is consistent across Oracle, FGMT, and GMT, with the largest gap between transfer methods and standard training observed in the first ten epochs, after which standard training gradually catches up. Naive transfer, by contrast, shows only modest acceleration and occasionally underperforms standard training in the very early epochs, illustrating that unaligned parameter transfer can mislead optimization rather than help it. Small quantitative differences with the original Figure 6 are observed for the GMT curve, where our replication yields slightly lower accuracy in mid-training epochs; we attribute this to the use of the linear trajectory approximation, which reduces the diversity of gradient information available to GMT’s greedy alignment procedure.

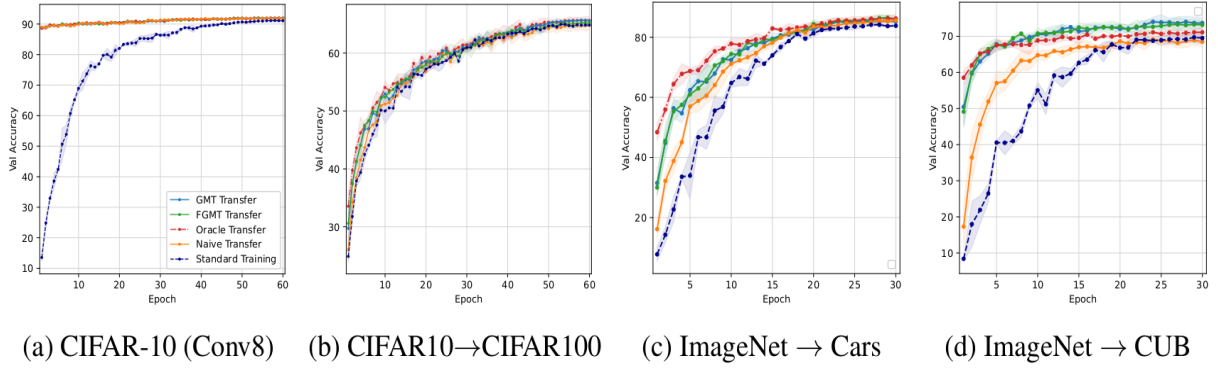


Figure 3: Validation accuracies during subsequent training of transferred parameters.

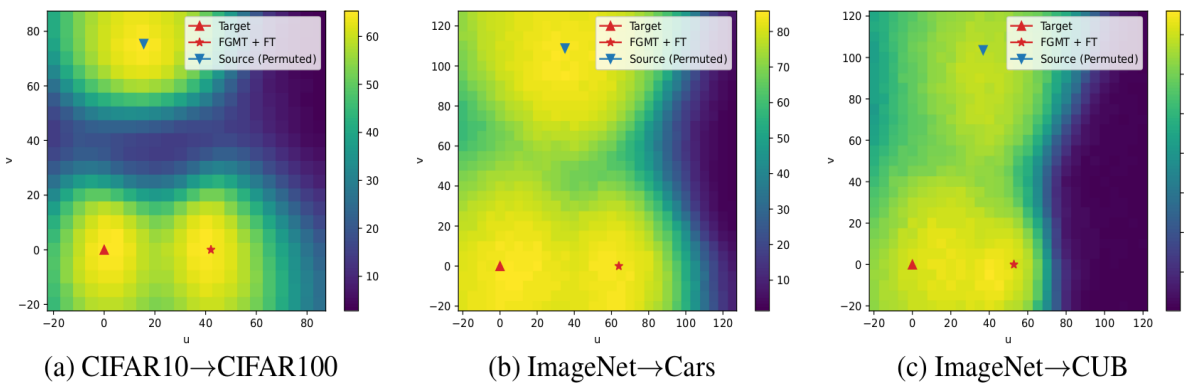


Figure 4: Inheritance of basin in loss landscape.

4.3 Basin Inheritance and Generalization

The loss landscape analysis confirms the basin inheritance hypothesis: the source parameters after permutation alignment, the FGMT-transferred parameters, and the target fine-tuned parameters all reside within the same flat loss basin, with no sharp barriers along any of the three pairwise linear interpolation paths connecting them. This geometric result explains why FGMT-initialized training converges faster: the target network does not need to escape a poor initialization basin, as it is already positioned within the correct one from the start. The generalization inheritance experiment further shows that FGMT initialized from ImageNet-10% and transferred to an ImageNet-Full target achieves both higher validation accuracy and a lower generalization gap than standard training from ImageNet-10% alone, demonstrating that the transferred trajectory carries genuine inductive biases that improve generalization beyond what the reduced dataset would provide by itself. A minor divergence is observed for the Cars dataset, where the generalization gap of FGMT (ImageNet-10% to ImageNet-10%) is slightly larger than reported in the original paper, likely due to the reduced diversity of the linearly approximated source trajectory compared to the actual nonlinear training path.

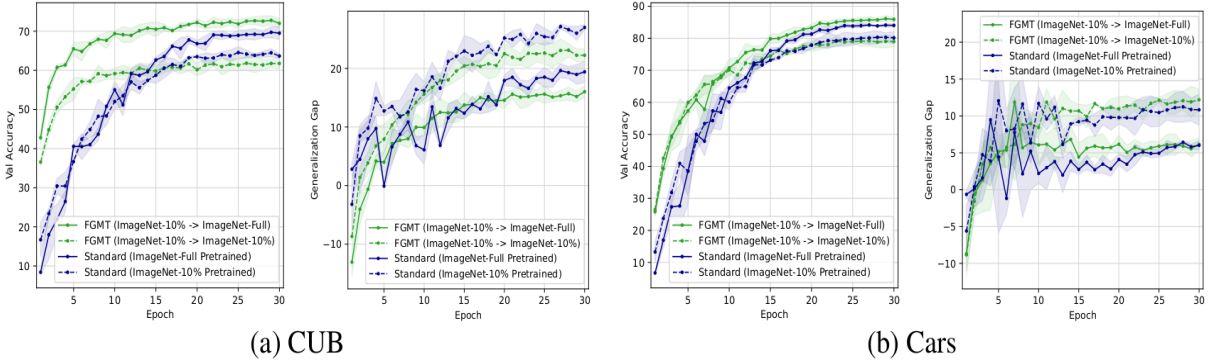


Figure 5: Inheritance of generalization ability.

5 Discussion and Critical Analysis

5.1 Conformity with the Original Article

The qualitative ordering $\text{Naive} \leq \text{GMT} \leq \text{FGMT} \leq \text{Oracle}$ is reproduced consistently across all evaluated configurations, and the validation of Assumption (P) on MNIST confirms that the cosine similarity $\rho_t = \langle \pi^* g_t^1, g_t^2 \rangle / (\|\pi^* g_t^1\| \cdot \|g_t^2\|)$ between permuted source increments and target increments is substantially higher than for the identity permutation. For hidden dimensions of 64 and 128, we observe values consistently above 0.7, confirming that wider networks exhibit stronger alignment properties as expected from the larger symmetry group. The pre-trained initialization results are particularly convincing: FGMT achieves meaningful accuracy from the very first trajectory timestep on Cars, CUB, and CIFAR-100, reproducing the qualitative finding that ImageNet pre-training imposes a strong inductive structure that makes gradient alignment robust across independently trained networks. The basin inheritance finding is also cleanly reproduced, with the three model checkpoints lying within a shared flat basin, and the generalization

inheritance experiment confirms that FGMT (ImageNet-10% to ImageNet-Full) achieves the lowest generalization gap among all four evaluated methods.

5.2 Impact of the Linear Trajectory Approximation

A key methodological deviation from the original paper concerns the source trajectory representation. While Chijiwa et al. use actual training checkpoints saved at each epoch, our replication approximates the source trajectory by linear interpolation $\theta_t^1 = \theta_0^1 + \frac{t}{T}(\theta_T^1 - \theta_0^1)$, which introduces a systematic bias that affects all transfer methods. The true training path is highly nonlinear: it exhibits rapid loss descent in early epochs, plateau phases near saddle points, and sharp directional changes at learning rate drops; all of this temporal structure is absent in the linear approximation, which spreads the total parameter displacement uniformly across the T timesteps regardless of the actual dynamics. The practical consequence is a systematic underestimation of the alignment quality achievable by FGMT: when all source gradient pairs carry similar directional information, the global aggregation advantage of FGMT over GMT is reduced, and both methods yield lower transferred accuracies than they would with real checkpoint trajectories. Our results (linear vs. actual trajectory transfer on CIFAR-10) corroborate this analysis, showing that actual checkpoint trajectories yield higher and more stable transferred accuracies across all timesteps, with the gap being most pronounced at early timesteps where the true training dynamics deviate most strongly from linearity.

5.3 Computational Limitations and Reproducibility

The full replication revealed substantial computational challenges that are not apparent from reading the paper. Running GMT, FGMT, Naive, and Oracle for three runs each on ResNet-18 configurations on CPU requires between one and two days per experiment, and the full execution of all notebook cells — including pre-training, transfer, subsequent training, and figure generation — spans approximately two weeks of continuous CPU computation, far exceeding the implicit assumptions of the paper which was conducted on multi-GPU infrastructure. The Jupyter notebook environment proved unstable for long-running ResNet-18 experiments, with the kernel being killed by the operating system out-of-memory manager after exhausting the available 15 GiB of physical RAM; this required activating a 16 GiB swap partition and migrating critical experiments to standalone Python scripts to bypass the kernel memory limits. These reproducibility challenges highlight a significant gap between the computational resources implicitly assumed by the paper and those available in a standard academic setting, and they underscore two practical lessons: first, intermediate state dict checkpointing should be built into transfer experiments from the start, as rerunning a one day ResNet-18 experiment from scratch due to a kernel crash is costly; second, a faithful replication of trajectory transfer at the scale of ImageNet and ResNet-18 requires GPU resources that should be considered a hard prerequisite rather than an optional convenience.

6 Conclusion

This replication study confirms the central thesis of Chijiwa et al. (2023): the learning trajectory of a neural network contains transferable information that meaningfully accelerates the training of an independently initialized target network, far beyond what any single parameter snapshot could provide. By aligning source and target trajectories through gradient-based permutation matching, FGMT consistently places the target network in a favorable region of the loss landscape, yielding faster convergence, inherited basin geometry, and improved generalization under data-scarce pre-training conditions, across architectures ranging from small MLPs to ResNet-18 and datasets ranging from MNIST to ImageNet-scale fine-tuning. Our replication reproduces all qualitative findings of the original paper and the majority of quantitative trends, with minor deviations attributable primarily to the linear trajectory approximation and CPU-only execution constraints rather than to any fundamental algorithmic discrepancy. The framework is theoretically well-grounded in permutation symmetry and Linear Mode Connectivity, the empirical evidence for Assumption (P) is strong and consistent across all configurations, and the practical benefits of FGMT over both Naive transfer and standard training are robust and reproducible. Future work should investigate whether lightweight checkpoint-free trajectory representations can replace the linear approximation without incurring full checkpoint storage costs, explore the extension of trajectory transfer to transformer architectures where the permutation symmetry group has a fundamentally different structure from that of MLPs and CNNs, and examine whether the observed generalization inheritance effect scales to larger pre-training data regimes and more challenging downstream tasks.